Screening EIP-7702 Delegates under an Execution-Audited Adaptive Adversary

***Abstract*—EIP-7702 lets an Ethereum externally owned account delegate execution to smart-contract code, creating a security decision over a program the adversary authored, at a moment when history, reputation, and verified source are frequently unavailable. We ask whether runtime bytecode alone supports screening at that boundary, and we show that the answer depends entirely on how the screener is evaluated. On an audited, family-disjoint benchmark of 2,190 delegates in 790 bytecode-similarity families, a 181k-token hierarchical attention model and a 15-feature logistic regression are statistically indistinguishable on clean bytecode (ΔAUPRC -0.021, 95% CI $[-0.078, +0.035]$), so clean metrics cannot select between them. We therefore evaluate a score-aware adaptive adversary that composes bytecode transformations under a 64-query budget, and—critically—we audit the transformation space itself: an independent real-EVM audit across 120 distinct bytecode families finds flooding (3464/3464), metadata rewrite (866/866), and neutral instruction insertion (866/866) preserve externally visible execution behaviour on every audited call. Restricting the adversary to that audit-supported action space, the two clean-equivalent screeners separate decisively: across three seeds, 1,503 paired observations and 209 families, the emulator suffers +0.370 higher attack success (95% CI $[+0.208, +0.520]$), and end-to-end robust recall—the fraction of positives detected cleanly *and* still detected after the strongest attack—is 0.679 versus 0.378. Widening to a broader stress-test space that includes transformations with weaker preservation evidence adds little: only 12–13% of its successes are unreachable within the audited space at the same budget. Parameter-matched controls and direct attention-mass measurements attribute the advantage to selective aggregation rather than capacity. We conclude that adaptive, preservation-aware evaluation should be standard practice for learned security screening over attacker-authored programs.**

***Index Terms*—EIP-7702, Ethereum, smart contract security, bytecode analysis, adversarial robustness, execution preservation, trust in delegated execution, security evaluation methodology**

I. Introduction

Ethereum separates externally owned accounts (EOAs), controlled by user keys, from smart contracts carrying executable logic. EIP-7702 dissolves that separation by letting an EOA delegate execution while keeping its address [1]. This enables transaction batching, gas sponsorship, and programmable authorization, but it also transfers execution authority over an account's assets, approvals, and transaction behaviour to third-party code. Authorizing an unsafe delegate is a trust decision taken when the usual bases for trust are frequently absent by construction: a delegate deployed minutes earlier has no history, no reputation, and often no verified source, while its runtime bytecode is already retrievable on chain. Figure 1 shows this window.

Two properties make this setting unusual. First, the artefact being screened is *authored by the adversary*, so any screener faces an attacker who can rewrite its input for little more than deployment gas. Second, the decision is interactive: it must be made in the moment before a signature. A screener that is accurate on unmodified code but yields to cheap edits provides no protection against the adversary it exists to stop, and a trust signal an adversary can manufacture is worse than no signal, because absence of warning reads as assurance.

This paper's central finding is that the evaluation protocol, not the model, determines what one concludes. We show that on clean bytecode a 15-feature logistic regression over hand-coded opcode and control-flow features is statistically indistinguishable from a hierarchical attention model (ΔAUPRC -0.021, 95% CI $[-0.078, +0.035]$), so clean classification metrics cannot justify choosing one over the other. Under a score-aware adaptive adversary the two separate by a wide, statistically supported margin.

The obvious objection to any such robustness claim is that the transformations may not preserve what the code actually does. We address this directly rather than by caveat. We audit each transformation class against a real EVM and separate an *audit-supported* action space—flooding, metadata rewrite, neutral instruction insertion—from a broader stress-test space that additionally rewrites embedded addresses and function selectors, for which preservation evidence is materially weaker. The audit covers 120 distinct bytecode families and 5,196 transformed executions, with no observed preservation failure in any of the three audit-supported classes. The headline security result is then stated over the audited space, where it is best evidenced, and we show separately that the broader space adds only modestly to attack success.

We present AuthGuard-7702, a bytecode-only screening framework producing a calibrated score and warning tier, built around AuthGuard-Seq: a compact hierarchical model that disassembles runtime bytecode, splits the opcode stream into 256-token chunks, encodes local order with shared convolutions, and aggregates chunk evidence with learned attention. The reference configuration uses 63,266 active parameters, processes up to 16,384 opcodes, and consumes no transaction history, source code, address identity, or reputation metadata.

A second methodological concern is dependence among contract samples. Code duplication inflates measured performance of learned models of code [10], and contract corpora contain redeployments and near-duplicate variants that let row-level random splits place related programs on both sides of a split [11]. We audit the source data, build bytecode-similarity families, keep identical and related bytecode within one fold, and exclude observations with unreliable source verdicts.

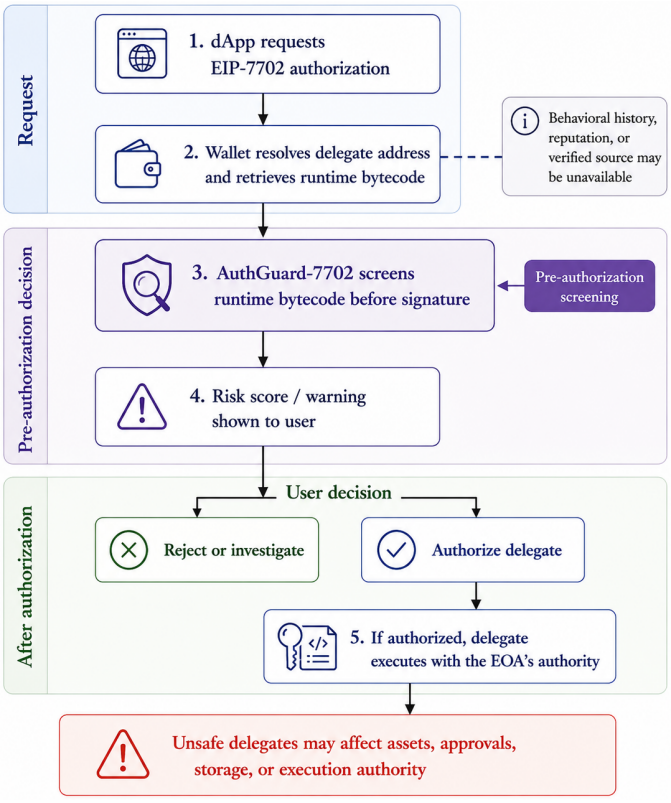

Fig. 1. EIP-7702 authorization flow and the pre-signature screening window.

The study makes four contributions.

- **An adaptive evaluation methodology with independent preservation auditing.** We contribute a query-budgeted, score-aware adaptive protocol over attacker-controlled EVM bytecode—composing transformations under a bounded byte budget with enforced structural validity—paired with a real-EVM execution-preservation audit across 120 bytecode families that partitions the action space by the strength of its behavioural evidence. The pairing is the contribution: robustness results are reported over an action space that has been independently audited, not merely asserted.
- **Evidence that adaptive evaluation changes model selection.** Clean evaluation leaves a cheap interpretable emulator statistically tied with the learned model. Under the audit-supported adaptive adversary the paired difference is $+0.370$ with 95% CI $[+0.208, +0.520]$ across three seeds, and robust recall is 0.679 versus 0.378. Fixed perturbation testing recovers only part of this gap, so it materially understates the security-relevant separation.
- **A mechanism finding.** Parameter-matched controls and direct measurement of attention mass show that selective aggregation, rather than capacity, is associated with resistance to attacker-controlled dilution and composition.
- **Operational evidence.** We report calibrated behaviour on an independently collected later temporal population of 752 delegates with explicit family-overlap auditing, and a measured cost boundary against a reference decompiler that motivates millisecond screening ahead of selective deeper analysis.

We claim no architectural novelty. Every component of AuthGuard-Seq is standard; the contribution is the adversarial requirement that motivates the design, the audited evaluation that isolates which mechanism satisfies it, and the negative controls that bound what the model adds.

II. Background and Related Work

A. EIP-7702 and the Authorization Boundary

EIP-7702 introduces authorization tuples through which an EOA delegates execution while keeping its address. For each valid tuple a delegation indicator formatted as `0xef0100‖address` is written to the authorizing EOA's code field, and subsequent code-executing operations resolve and run the code at the referenced delegate [1]. The object worth screening is therefore the delegate's runtime bytecode, not the 23-byte indicator. Once authorized, that code executes in the EOA's context, which makes the pre-signature moment the natural automation point.

Huang et al. combine transaction analysis with cross-contract static analysis to characterize EIP-7702 malicious behaviour [2]; Qi et al. study delegation-based phishing and protocol defenses [3]. Neither investigates learned screening of proposed delegate bytecode before authorization.

B. Learning over Contract Code, and How Robustness Is Evaluated

Early learning-based contract security work reports clean classification metrics: ContractWard uses opcode bigrams with conventional learners [5], Eth2Vec learns contract-wide bytecode representations [6], PhishingHook evaluates bytecode and opcode representations for phishing-contract classification [4], and later work combines bytecode features with control-flow-graph analysis [12]. Surveys identify representation, dataset construction, and evaluation design as recurring determinants of reported performance [13].

More recent systems do evaluate robustness, and it would be inaccurate to characterise the field as reporting clean metrics alone. FinDet detects adversarial exploiter contracts from bytecode alone at the pre-deployment stage, using LLM-derived semantic interpretation with uncertainty estimation, and reports robustness to unseen attack patterns and to feature obfuscation [7]. ContractShield targets multi-label vulnerability detection in *obfuscated* contracts, fusing source code, opcode sequences, and control-flow graphs, and evaluates resilience under obfuscation applied at both source and bytecode level [8]. ORACAL combines control-flow, data-flow, and call graphs with causal reasoning and reports an attack success rate of approximately 3% under adversarial attack against baselines in the 10.9–18.7% range [9].

Our setting differs along three axes, and we state the distinction narrowly. *Task*: FinDet screens contracts that attack other contracts, while ContractShield and ORACAL detect vulnerabilities in the contract under analysis; we screen a proposed delegate for the authority it would acquire over a user's

TABLE I
POSITIONING AGAINST REPRESENTATIVE PRIOR WORK

Work	EIP-7702	Bytecode only	Pre-auth. screening	Adaptive attacker	Preservation audited
Huang et al. [2]	✓	–	–	–	–
Qi et al. [3]	✓	–	–	–	–
PhishingHook [4]	–	✓	–	–	–
Eth2Vec [6]	–	✓	–	–	–
ContractWard [5]	–	✓	–	–	–
FinDet [7]	–	✓	–	fixed	–
ContractShield [8]	–	–	–	fixed	–
ORACAL [9]	–	–	–	fixed	–
AuthGuard-7702	✓	✓	✓	**score-aware**	✓

"Adaptive" distinguishes a predefined perturbation set (*fixed*) from an attacker that observes the screener's scores and searches over compositions (*score-aware*). "Preservation audited" denotes independent execution-level validation of the transformations used in the robustness evaluation.

account, a decision taken by the account holder before signing. *Inputs*: ContractShield consumes source code alongside opcode and graph modalities and ORACAL consumes graph representations, whereas the pre-authorization setting supplies only deployed runtime bytecode; FinDet shares this bytecode-only constraint. *Adversary*: to our knowledge these evaluations apply predefined obfuscation or perturbation sets, and do not report a *score-aware* attacker that repeatedly queries the screener and adaptively composes transformations to drive its score below the deployment threshold; nor do they report execution-preservation auditing that separates transformations by the strength of their behavioural evidence. We therefore do not claim the first adversarial evaluation of a contract classifier. We claim the narrower and precise contribution of a query-budgeted adaptive adversary over an independently execution-audited transformation space, applied to the EIP-7702 pre-authorization decision. Table I summarises the positioning.

C. Static Analysis and Security-ML Evaluation

Learned screening complements rather than replaces semantic analysis. Gigahorse decompiles EVM bytecode for declarative analysis [14] and Securify derives semantic dependencies and checks security patterns [15]. Our reference labels originate from a decompiler-based rule [2]; our question is whether that signal can be approximated from opcode sequences fast enough for interactive screening, and whether such an approximation survives an adversary. TESSERACT shows how dependence and distribution bias inflate security-classification results [11]; we isolate identical and similar bytecode families across folds and interpret transformed executable inputs only within their demonstrated preservation guarantees, following problem-space adversarial-ML guidance [16].

III. PROBLEM FORMULATION AND THREAT MODEL

A. Pre-Authorization Screening Task

Let b be the normalized runtime bytecode at a proposed delegate address and $T(b)$ its opcode-token sequence. A trained model f_θ produces a risk logit that is temperature-scaled on validation data [17]:

$$s(b) = \sigma\left(\frac{f_\theta(T(b))}{T_{\text{cal}}}\right), \qquad T_{\text{cal}} > 0, \qquad s(b) \in [0,1]. \quad (1)$$

The positive class consists of delegates flagged by the source study's static-analysis rule, which identifies decompiled reachability from standardized inbound hooks—`receive()`, `fallback()`, and token-reception callbacks—to predefined value-moving external-call patterns [2]. The negative class is delegates from the same pool that were not flagged. Because an unflagged verdict does not establish benign semantics, the task is screening for a source-analyzer-identified structural condition, not proving malicious intent, exploitability, or safety. We write "source-flagged" rather than "malicious" throughout.

This label provenance is a genuine limitation, and the paper's design responds to it structurally rather than rhetorically. Our headline result is a *relative* comparison among screeners evaluated against a shared label under a shared, audited adversary. Whatever the label denotes, the ranking of screeners under attack is a statement about the screeners and does not require the label to be adjudicated ground truth. This is the principal reason the paper is organised around adversarial robustness rather than absolute detection quality.

From negative validation scores we derive thresholds $\tau_1 \geq \tau_5 \geq \tau_{10}$ for nominal 1%, 5%, and 10% false-positive-rate (FPR) operating points, mapping scores to HIGH, WARNING, CAUTION, and LOW-OBSERVED-RISK. All attack results below use the nominal 5% operating point.

B. Information Boundary

The model observes only the ordered opcode sequence derived from runtime bytecode. PUSH immediate operands are skipped, so the representation captures opcode identity and order rather than embedded constants. The model receives no transaction history, balances, approvals, graph or address identity, source code, family or fold identifiers, or any field used to construct the reference labels. AuthGuard-7702 is an advisory first-stage screener; it does not resolve proxy state, simulate transactions, infer off-chain intent, or certify safety.

C. Adversary Model

The adversary wants a delegate whose runtime code exhibits the reference condition to score below the wallet's warning threshold, and **authors the bytecode**. We grant:

- *Score access.* The adversary may query the screener and observe its output, bounded at 64 queries per contract.
- *Edit capability.* The adversary may append content, rewrite metadata, insert neutral instructions, and—in the broader stress-test space only—rewrite embedded address and selector immediates, composing up to four actions with at most one flooding action.
- *A cost budget.* Appended content is capped at $2\times$ the original executable size, so the transformed program is at most approximately $3\times$ the original size. Empirically, Flood-200% candidates have a median final-to-original size ratio of 2.962 (p95 2.986, maximum 3.001, $n = 13{,}086$).

We do *not* grant white-box gradient access or the ability to retrain the defender's model. Score access with a bounded query budget is the appropriate model for a screening service,

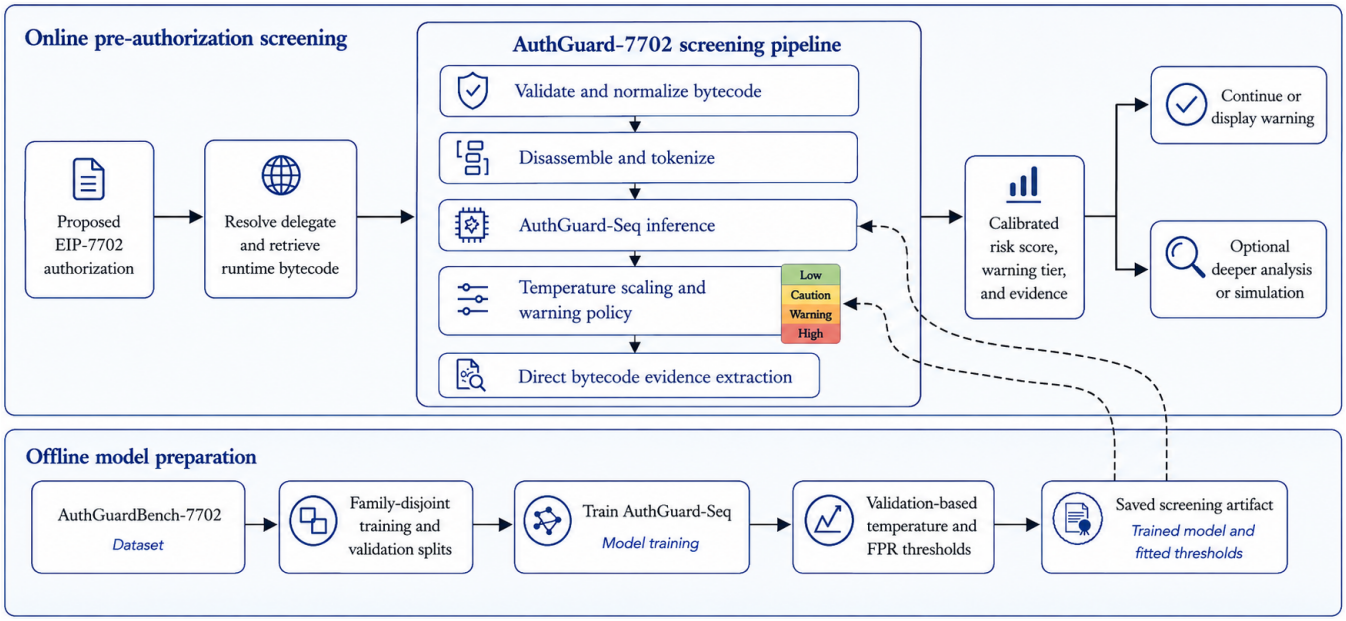

Fig. 2. AuthGuard-7702 system overview: the online screening path at the authorization boundary and the offline family-disjoint training and calibration path.

and is deliberately conservative for a screener shipped inside a wallet client, where a determined adversary could extract parameters and mount white-box attacks; results below are correspondingly lower bounds on evadability for client-side deployment.

D. Attack Tiers and Their Evidentiary Status

The transformations available to the adversary do not carry equal behavioural evidence, and treating them as interchangeable would be the central weakness of any robustness claim here. We therefore partition the action space into three nested tiers by the strength of that evidence, audit the tiers independently (Section VI-B1), and report the primary security result over the audited space.

- Tier A — Appending supported following bytecode. deterministic donor bytes at 25%, 50%, 100%, or 200% of the source executable length. Because only one flooding action is permitted, Tier A is evaluated as an exhaustive per-source oracle over the four levels rather than as a compositional search.
- Tier B — Flooding, unrestricted composition, and adversarial instruction insertion, composed adaptively. Every transformation class in Tier B carries independent execution-preservation support. **Tier B is the paper's headline adversary.**
- Tier C — Tier B plus restricted rewriting of embedded address and function-selector immediates. These additional primitives have materially weaker preservation evidence and Tier C is therefore reported as a structural-validity stress test, not as the foundation of the primary claim.

Tiers B and C run two search strategies under an identical 64-query budget: a seeded *random search* over sampled action sequences and a score-guided *beam search* with width 4 and depth 4. Both are deterministic given the seed. We report the maximum over strategies, so a weak guided search cannot flatter the defence.

Three constraints keep the attack honest. Every candidate must be **structurally valid**—it must disassemble and satisfy an opcode-skeleton preservation check against the source. Every candidate must respect the byte budget above. Flooding donors are drawn from partition-isolated pools such that a donor never shares the recipient's bytecode family and never crosses the train/validation/test boundary, with per-segment provenance recorded.

An attack *succeeds* on an observation if the clean score is at or above the model's own validation-derived 5% FPR threshold and the adversarial score falls below it. Attack success rate (ASR) is computed over observations the model detects cleanly, since a source it never flagged cannot be evaded.

IV. AuthGuard-7702

A. Screening Workflow

Figure 2 places AuthGuard-7702 at the authorization boundary. A wallet resolves the proposed delegate address, retrieves runtime bytecode, and invokes the local path before the user signs. The pipeline normalizes and disassembles the bytecode, runs AuthGuard-Seq, applies validation-fitted calibration and thresholds, and returns a score, warning tier, and auxiliary bytecode evidence. Offline, the training path constructs family-disjoint folds, trains the model, and fits the temperature and FPR thresholds on validation data only.

B. Design Under an Adversarial Requirement

The design follows from the adversary model rather than from architectural ambition. Because the attacker controls program length and can append arbitrary content cheaply, the aggregation step must not let appended bytes dilute or dominate the risk signal. This yields three requirements, each tied to an attack and each tested in Section VI-D:

1) *Full-stream coverage.* Fixed-window models can be evaded by pushing the payload past the window, so the model must see the whole program rather than a prefix.
2) *Local order sensitivity.* Histogram and n-gram representations discard instruction order, which order-preserving rewrites exploit; local convolutions retain short ordered motifs.
3) *Length-robust selective aggregation.* Uniform pooling over a longer program dilutes evidence in proportion to appended content. Learned attention can concentrate on the informative region largely independently of how much padding surrounds it.

C. Hierarchical Opcode-Sequence Model

AuthGuard-Seq begins with deterministic linear-sweep EVM disassembly. Recognized opcodes map to stable token IDs, undefined bytes use a dedicated token, and padding uses a separate ID. PUSH immediates are skipped by instruction width. Tokenization does not terminate at the first linear `STOP`, so trailing regions are not discarded because of an early stopping opcode. The sequence is split into chunks of length $L = 256$, at most $K = 64$ chunks. The longest benchmark program has 16,081 opcodes and fits the 16,384-token capacity, so every benchmark contract is processed in full.

Each chunk is embedded into 32 dimensions, processed by a kernel-5 convolution followed by a kernel-3 dilated convolution with dilation 2, and reduced by masked maximum pooling to a 64-dimensional h_k. Learned attention aggregates valid chunks:

$$\alpha_k = \frac{\exp(w^\top h_k)}{\sum_{j=1}^{K_b} \exp(w^\top h_j)}, \qquad h = \sum_{k=1}^{K_b} \alpha_k h_k, \tag{2}$$

where w is learned and K_b is the number of non-padding chunks. The aggregate passes through a 128-dimensional layer with GELU, dropout, layer normalization, and a binary risk head. The logit is temperature-scaled before thresholds apply. Figure 3 summarizes the architecture.

Parameter accounting: AuthGuard-Seq is the sequence-only configuration of a fusion architecture that also defines n-gram and dense-structural branches. Those branches are constructed but receive no gradient in this configuration, so a naive parameter sum overstates the model and reports an identical count for configurations that differ. We report parameters on the trained forward path, verified by checking which tensors receive gradient: **63,266** for the reference AuthGuard-Seq configuration, against 108,225 for the n-gram variant and 75,595 for the dense-structural variant.

TABLE II
BENCHMARK POPULATIONS. THE AUDITED-IMPLEMENTATION REGISTRY IS A SEPARATE RESOURCE AND IS NOT INCLUDED IN THE 3,082 TOTAL.

Population	n	Reference status
Primary evaluation	2,190	727 flagged / 1,463 unflagged
External benign control	797	Benign-labeled
Qualitative production control	5	Curated legitimate implementations
Excluded uncertain input	90	Unreliable source verdict
Benchmark total	**3,082**	
Separate resources, not part of the benchmark total		
Audited-implementation registry	45	8 projects, 30 unique bytecodes
Live temporal holdout	752	Unlabeled, 669 unique bytecodes

V. BENCHMARK AND METHODOLOGY

A. Audited Benchmark Construction

AuthGuardBench-7702 contains 3,082 audited observations partitioned before evaluation (Table II). The primary population derives from the EIP-7702 delegate dataset and analyzer outputs of Huang et al. [2]; after auditing it holds 2,190 delegates, 727 source-flagged and 1,463 unflagged. External controls contribute 797 benign-labeled general Ethereum contracts from the PhishingHook-derived source [4], and five curated production EIP-7702 implementations serve as in-benchmark qualitative controls. A further 90 observations are excluded because their original unflagged verdicts came from truncated or failed bytecode retrieval: 89 negatives truncated at a spreadsheet cell limit and one timeout string stored in place of bytecode.

Separately from the benchmark, we maintain an *audited-implementation registry* of named production EIP-7702 and account-abstraction deployments: 8 projects, 45 address rows across chains, 30 unique runtime bytecodes. The registry is not part of the 3,082 benchmark observations and is never pooled with them; it is used only for the qualitative check in Section VI-E.

Section VI-E adds a fourth population collected independently for this work: 752 screenable Ethereum delegates observed in a two-week window strictly later than the benchmark.

B. Families, Duplicates, and Splits

Runtime bytecode is linearly disassembled and represented as opcode 4-gram sets. Deterministic 128-permutation BLAKE2b MinHash signatures with a fixed 0.85 agreement threshold define similarity links, merged by union-find into 790 families. Identical bytecode with conflicting reference labels is quarantined before modeling.

We use five fixed family-disjoint outer folds. For test fold f, fold $(f + 1) \bmod 5$ is validation and the remaining three are training. No family or identical bytecode crosses folds. Configurations are repeated with seeds 7702, 7703, and 7704.

C. Frozen Models and Replication

Every attack tier for a given (model, seed, fold) is evaluated against the *same* frozen checkpoint, so tier differences cannot be confounded by retraining. Checkpoints were produced by the unmodified training procedure and persisted with weights,

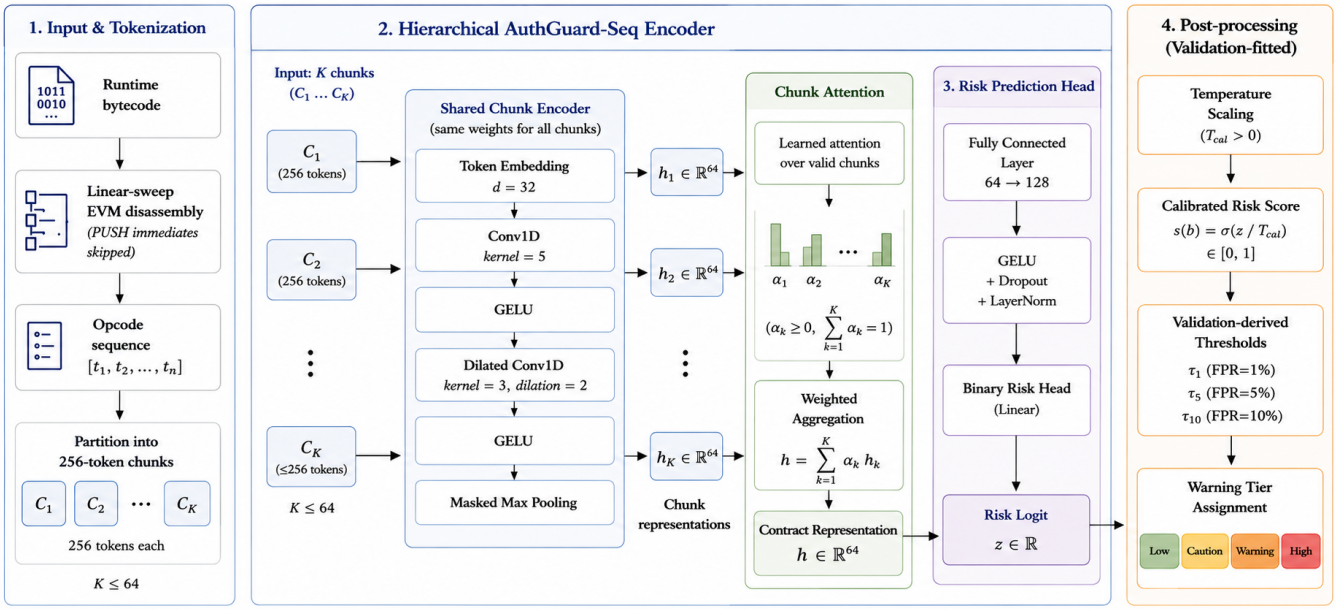

Fig. 3. AuthGuard-Seq: chunked opcode encoding, shared local convolutional encoders, and learned chunk attention. Selective aggregation is the mechanism isolated in Section VI-D.

calibration temperature, all three thresholds, environment, and content hashes.

As a replication check, the frozen models were re-evaluated under the original attack protocol. At seed 7702 the maximum absolute difference from the prior run across all models and metrics is 0.011; the two deterministic estimators—the logistic-regression emulator and the gradient-boosted model—reproduce exactly ($\Delta = 0.0000$ on clean detection, beam ASR, and random ASR), and the residual drift is confined to the two stochastically trained neural models. A data, fold, preprocessing, or hyperparameter mismatch would have moved the deterministic estimators too, so the tier evaluation rests on models generated by the same underlying pipeline.

D. Baselines, Training, and Statistics

The clean comparison evaluates seven predefined configurations under identical folds, validation rotation, seeds, early stopping, calibration, and operating points. Neural models use class-weighted binary cross-entropy, AdamW at learning rate 10^{-3}, weight decay 10^{-4}, gradient clipping at 5, at most 30 epochs, and validation-AUPRC early stopping with patience 5. Temperature scaling is fit on validation logits only and thresholds derive only from negative validation examples. Test labels never influence model selection, calibration, or thresholds.

For inferential comparisons the bytecode family is the resampling unit: families are resampled with replacement and the same multiplicities apply to both members of a pair. **Replicate counts differ by experiment and are stated per table**: 10,000 for the clean and attack comparisons, 2,000 for the mechanism ablation, and 1,998 for the cheap-baseline controls.

Marginal versus paired statistics: Two distinct quantities appear below and are never interchanged. A *marginal* ASR is computed over one model's own cleanly-detected observations. A *paired* contrast is computed over the intersection of observations that *both* compared models detect cleanly, and is the mean of per-observation differences on that shared population. The two differ because the models do not detect the same sources.

VI. Evaluation

Five questions. RQ2 is the central one.

- **RQ1** What does clean evaluation actually distinguish?
- **RQ2** Does the robustness advantage survive an execution-preservation-supported adversary?
- **RQ3** What does the broader, weaker-evidence action space add?
- **RQ4** Does the mechanism explain the difference?
- **RQ5** How does the screener behave operationally?

A. RQ1: What Clean Evaluation Distinguishes

Table III reports clean family-disjoint results. AuthGuard-Seq ranks first at 0.924 ± 0.014 AUPRC and 0.833 ± 0.016 Recall@5% FPR. Stated operationally, that recall means roughly one in six source-flagged delegates is not surfaced at the recommended operating point even on unmodified bytecode—a first-stage triage layer, not a gate.

A cheap baseline is statistically tied: We deliberately construct a strong trivial competitor: an L2 logistic regression over 15 hand-coded opcode and control-flow features, including a conservative and an over-approximate variant of "does a fallback path reach an external call." It reaches 0.911 AUPRC against AuthGuard-Seq's 0.924, and

TABLE III
CLEAN FAMILY-DISJOINT PERFORMANCE (3 SEEDS × 5 FOLDS, MARGINAL). PARAMETERS FOR FUSION CONFIGURATIONS ARE ACTIVE-PATH COUNTS.

Model	Params	AUPRC ↑	AUROC ↑	Brier ↓	R@5% ↑
AuthGuard-Seq	63,266	**.924±.014**	**.963±.011**	**.072±.012**	**.833±.016**
Flat CNN	154,177	.885±.010	.937±.007	.099±.004	.712±.024
Hist.+4-gram XGB	–	.833±.004	.907±.002	.127±.004	.615±.015
Neural n-gram	108,225	.810±.007	.880±.020	.125±.001	.654±.029
BiGRU	108,225	.679±.098	.815±.069	.163±.027	.379±.113
Dense structural	75,595	.637±.018	.780±.022	.182±.009	.331±.023
Transformer	425,217	.563±.031	.730±.019	.208±.013	.239±.054

TABLE IV
EXECUTION-PRESERVATION AUDIT. 120 DELEGATES, ONE PER BYTECODE FAMILY, REAL-EVM TRACES, WILSON 95% INTERVALS.

Transformation class	Families	Calls	Preserved	Wilson 95%
Tier A / Tier B — audit-supported				
Flooding (25–200%)	120	3,464	**3,464**	[0.9989, 1.0000]
Metadata rewrite	120	866	**866**	[0.9956, 1.0000]
Neutral insertion	120	866	**866**	[0.9956, 1.0000]
Tier C only — weaker evidence				
Address rewrite	10	100	23	not promoted

the paired family-clustered difference is -0.021 with 95% CI $[-0.078, +0.035]$, which **spans zero**. It also runs faster: 2.626 ms versus 4.121 ms median. A single boolean—over-approximate fallback-to-external-call reachability—achieves recall 0.996 at precision 0.760 on its own.

We report this as a result, not a caveat. On clean bytecode the reference condition is cheaply recoverable, and clean accuracy alone does not justify a learned sequence model. A memorization control confirms the split is sound rather than the task being trivially near-duplicate: k-NN over MinHash signatures reaches only 0.612 AUPRC, -0.302 against AuthGuard-Seq with CI $[-0.380, -0.226]$.

The rest of the paper follows from this: if clean metrics cannot select between a 15-feature regression and a hierarchical neural model, the selection must be made on a criterion that matters at an attacker-controlled boundary.

B. RQ2: Robustness under an Execution-Audited Adversary

1) Execution-preservation audit: Before reporting robustness we establish what the transformations do. Each class was audited against a real EVM: for each delegate we execute a calldata suite—empty calldata, the zero selector, and up to eight selectors discovered in the bytecode—against the original and the transformed runtime, and compare externally visible behaviour. A call is *preserved* when failure status, return data, the set and count of external calls, the set of storage writes, and the log count all match. Delegates are sampled one per bytecode family for family diversity.

Table IV reports the outcome: **no preservation failure was observed in any of the three audit-supported classes**, across 5,196 transformed executions and 120 distinct families. The failure taxonomy is consequently empty. Coverage includes the inbound hooks the reference rule keys on—for flooding, 480 empty-calldata executions reaching `receive()`/`fallback()` and 480 zero-selector executions—and 788 flooding calls whose original trace actually reached an external call. Address-immediate rewriting preserved only 23 of 100 audited calls in the same harness and is therefore *not* promoted; it remains confined to Tier C.

We describe Tier B as an *independently audited, empirically execution-preservation-supported* action space. We do not claim formal semantic equivalence: the evidence is bounded empirical preservation over a finite calldata suite, not a proof.

TABLE V
MARGINAL ATTACK SUCCESS AND ROBUST RECALL BY TIER, AGAINST FROZEN CHECKPOINTS. LOWER ASR AND HIGHER ROBUST RECALL ARE BETTER. FAMILY-CLUSTERED 95% CIS, 10,000 REPLICATES. **SEED SCOPE DIFFERS BY MODEL AND IS STATED PER ROW.**

Model	Tier	Clean det.	ASR	95% CI	Robust recall	Seeds
AuthGuard-Seq	A	.8450	**.1492**	[.105, .203]	**.7189**	3
	B	.8450	**.1970**	[.148, .256]	**.6786**	3
	C	.8450	.2084	[.158, .268]	.6690	3
15-feature emulator	A	.8253	.3083	[.218, .413]	.5708	3
	B	.8253	**.5417**	[.427, .645]	**.3783**	3
	C	.8253	.5722	[.461, .675]	.3530	3
Flat CNN	A	.6836	.8109	[.718, .889]	.1293	1
	B	.6836	.9356	[.893, .972]	.0440	1
	C	.6836	.9477	[.914, .978]	.0358	1
Hist.+4-gram XGB	A	.6190	.8111	[.754, .866]	.1169	1
	B	.6190	.9844	[.967, .997]	.0096	1
	C	.6190	.9911	[.981, .998]	.0055	1

TABLE VI
CENTRAL PAIRED COMPARISON: EMULATOR MINUS AUTHGUARD-SEQ. THREE SEEDS, 1,503 PAIRED OBSERVATIONS, 209 BYTECODE FAMILIES. **PAIRED STATISTICS OVER THE SHARED CLEAN-DETECTED POPULATION; NOT THE MARGINAL VALUES OF TABLE V.**

Tier	Adversary	Emulator	AuthGuard	Difference	95% CI
A	flooding oracle	.3353	.1271	+.2083	[+.076, +.334]
B	**audited adaptive**	**.5369**	**.1670**	**+.3699**	**[+.208, +.520]**
C	full stress test	.5635	.1810	+.3826	[+.223, +.524]

2) Attack success across tiers: Table V reports marginal results per model and tier against frozen checkpoints. AuthGuard-Seq and the emulator were evaluated at three seeds; Flat CNN and the gradient-boosted model at seed 7702. Rows are never ranked across differing seed scopes.

3) The central comparison: The decisive comparison is between the two screeners that clean evaluation could not separate. Table VI reports the paired contrast over the intersection of observations both models detect cleanly.

Under the audit-supported Tier-B adversary, the clean-equivalent emulator suffers **0.370 higher attack success than AuthGuard-Seq, with a family-clustered 95% CI of** $[+0.208, +0.520]$ **across three seeds**. All three intervals exclude zero. Figure 4 shows the gap widening with adversary strength.

4) Robust recall: ASR is conditional on each model's own clean detections, which differ. We therefore also report *robust recall*: the fraction of *all* positive observations that are detected on clean bytecode *and* remain detected after the strongest attack in the tier. It was computed directly from source-level outcomes and agrees with clean $\times$ (1 − ASR) to numerical precision.

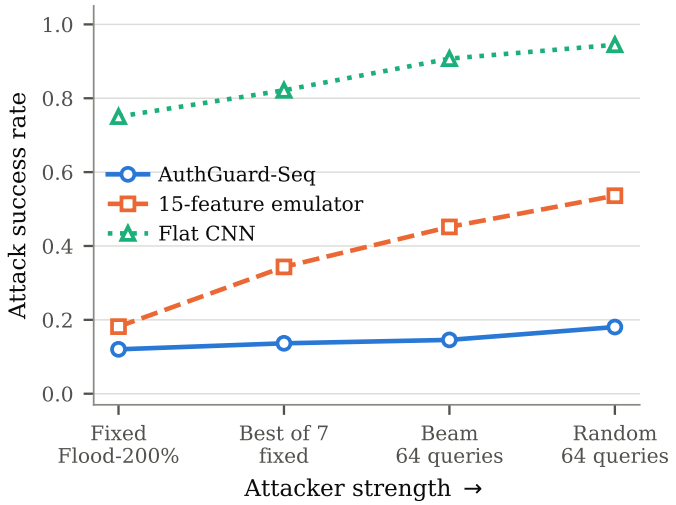

Fig. 4. Attack success against adversary strength for the historical stored-artifact comparison. Models that are close under a single fixed transformation separate under budgeted adaptive search.

Under the audited Tier-B adversary, **AuthGuard-Seq retains approximately 68% end-to-end robust recall (0.6786) against approximately 38% (0.3783) for the clean-equivalent emulator**; the flat convolutional and gradient-boosted screeners retain 0.0440 and 0.0096 respectively at seed 7702. Robust recall is the number a deployment cares about, because it charges the screener for both missed detections and successful evasions on a single common denominator.

C. RQ3: What the Broader Action Space Adds

Tier C adds address and selector rewriting, whose preservation evidence is weak. If the robustness separation depended on those primitives, the headline claim would rest on transformations we cannot support. It does not.

Moving from Tier B to Tier C raises the paired difference only from $+0.3699$ to $+0.3826$, and among Tier-C successful evasions the fraction *not* reproduced by Tier B within the same 64-query budget is 46/384 (12.0%) for AuthGuard-Seq and 137/1030 (13.3%) for the emulator; the corresponding figures for the flat convolutional and gradient-boosted screeners are 14/471 (3.0%) and 5/446 (1.1%).

We state the interpretation precisely: this does not show that the weaker-preservation primitives are logically necessary or unnecessary for any particular evasion. It shows that, for the evaluated 64-query attacker, the large robustness separation is already present under the independently audit-supported Tier-B action space, and expanding to the full space yields only a modest additional increase in attack success.

Adaptive composition versus fixed perturbation: A fixed multi-level flooding oracle already produces a statistically supported separation on the historical records: $+0.184$ with CI $[+0.035, +0.330]$, whereas a single Flood-200% transformation alone does not ($+0.082$, CI $[-0.047, +0.218]$). Under adaptive composition over the audited space the gap grows to $+0.370$. The accurate methodological conclusion is therefore not that fixed transformations are uninformative, but that **fixed perturbation testing materially underestimates the robustness gap that adaptive composition exposes**—by roughly a factor of two between the fixed flooding oracle and the audited adaptive adversary.

D. RQ4: Does the Mechanism Explain the Difference?

Parameter-matched controls: A separate controlled study matches five variants at roughly 30,000 parameters, isolating token budget, layout, and aggregation. On clean bytecode a flat encoder given the same 16,384-token budget is at least as good as chunk attention—the hierarchy contrast is -0.018 with CI $[-0.040, +0.018]$, spanning zero—so we do not claim clean superiority for hierarchy. Under flooding the ordering inverts: attention over mean pooling gives $+0.180$ AUPRC (CI $[+0.135, +0.232]$) and $+0.541$ Recall@5% (CI $[+0.474, +0.631]$), and chunking over flat gives $+0.098$ (CI $[+0.059, +0.154]$). Coverage alone is inconclusive in both conditions. The best clean model is the worst model under attack.

Attention mass: Because the model exposes its chunk weights, the aggregation argument can be measured rather than inferred. For all 727 held-out positives, scored by each fold's own checkpoint, we recorded the fraction of attention mass falling on chunks composed entirely of appended donor bytes. Uniform weighting has no learned parameters, so its mass on appended chunks is exactly their share of the chunk count; this is an *analytic reference*, not a separately trained measurement. Under Flood-200% appended content occupies 63.1% of chunks while attention places 37.6% of its mass there, a gap of 25.5 percentage points that widens monotonically with flooding volume (8.2, 14.7, 22.1, 25.5 points at 25%, 50%, 100%, 200%). This is consistent with the observed robustness advantage and provides mechanistic evidence that selective aggregation resists dilution by appended content. It is also partial: attention still assigns more than a third of its weight to content carrying no signal, which is consistent with a residual attack success rate well above zero.

Training cannot substitute for aggregation: A natural objection is that robustness might come from training data rather than design. Retraining each model with its training set doubled by flooding-augmented copies helps against the fixed transformation it was trained on but not against adaptive search: at seed 7702 the flat convolutional screener moves from .945 to .927 marginal random-search ASR and the gradient-boosted model from .986 to .940, both remaining far worse than clean-trained AuthGuard-Seq. The same intervention on AuthGuard-Seq reduces its marginal ASR from .1882 to .0623 at essentially no clean cost (.8404 to .8391), a paired reduction of -0.133 with CI $[-0.194, -0.086]$ over the 593 shared observations. Augmentation and selective aggregation are complementary rather than alternative. All figures in this paragraph are seed 7702 only, and the paired contrast is computed on the shared clean-detected population, not by subtracting the marginal values.

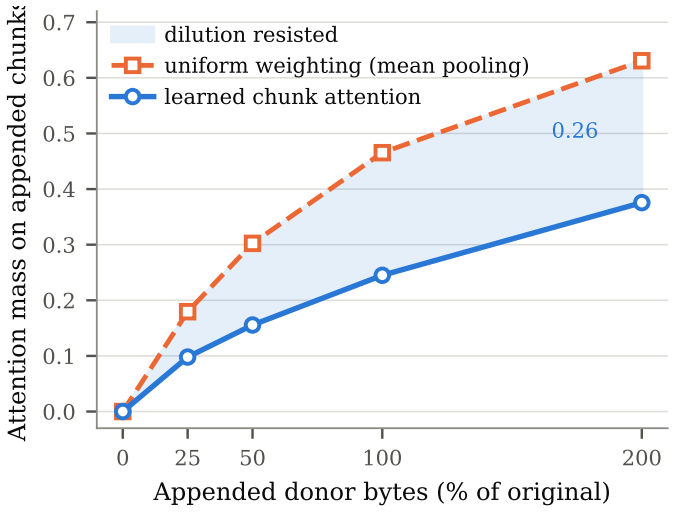

Fig. 5. Attention mass on appended donor chunks against the uniform-weighting analytic reference. All 727 held-out positives, each scored by its own fold's checkpoint.

E. RQ5: Operational Behaviour

External and production controls: Applying primary validation thresholds to 797 benign-labeled general Ethereum contracts yields FPR 0.015 ± 0.004, 0.065 ± 0.012, and 0.169 ± 0.021 at the nominal 1%, 5%, and 10% points. Across the audited-implementation registry, three of the eight named projects fall within primary training families; scoring current checkpoints on those would leak, so they are excluded from any aggregate and reported descriptively only. Of the remainder, none is flagged at the 5% point, though several fall in the CAUTION band at 10%. With five leakage-clean projects this is a qualitative check, not a false-alarm estimate.

A later temporal population: The benchmark carries no timestamp, so temporal generalization was previously untestable. We collected an independent population: scanning 100,444 Ethereum blocks yielded 1,335,391 authorization entries and 752 screenable distinct delegates first observed over a two-week window strictly later than the benchmark, with 669 unique runtime bytecodes.

Exact bytecode-hash overlap with the entire benchmark is zero. That is not sufficient evidence of novelty, and we audit it explicitly rather than claiming disjointness: applying the benchmark's own 0.85 MinHash family threshold, **70 of 752 (9.3%)** live delegates would join a training family, one at similarity 1.000. This is therefore an *independently collected later temporal population with explicit family-overlap auditing*, and we report leakage-clean figures over the 682 delegates below the threshold alongside the full population.

We assert no labels for this population, so its flag rate is not a false-positive rate—it is real mixed traffic containing genuinely risky delegates. The deployment-relevant figure is weighted by observed authorization volume, because users encounter delegates in proportion to how often they are authorized: the nominal 5% point flags 9.4% of live authorization volume and the nominal 1% point flags 5.9%. By distinct contract the same points flag 20.3% and 9.5% (leakage-clean: 19.9% and 8.8%), so population-specific recalibration is mandatory before deployment. The gap between per-contract and per-authorization rates arises because high-volume delegates are predominantly popular wallet-infrastructure implementations that score low.

Cost and staged triage: Across 1,500 complete screening calls on one CPU thread, the full local path has median latency 4.121 ms, p95 14.547 ms, and p99 21.429 ms. For the deeper-analysis stage we measured a pinned Gigahorse decompiler image on 60 deterministic family-distinct inputs: all 60 succeeded, with warm median 2.687 s and p95 5.618 s. The median ratio is 652×.

The value of this gap is not aggregate throughput. It is that the authorization decision is interactive and synchronous: a newly encountered delegate is a cache miss by construction, and a multi-second decompilation cannot sit in front of a user waiting to sign. Millisecond screening allows every proposed delegate to be assessed at signing time and expensive semantic analysis to be reserved for escalated cases. Three scoping statements are essential: this measures decompilation and lifting only, with **no downstream client rule executed**; it is not the analyzer of Huang et al.; and it is not a comparison of accuracy or substitutability. The speedup accrues to fast bytecode-level screening in general—the 2.626 ms emulator is also an instance—so it is not unique to AuthGuard-Seq.

VII. DISCUSSION

What the results support: Clean accuracy is achievable by very cheap models on this task, so it is a poor basis for choosing a screener. Robustness to an adversary who authors the bytecode is not cheaply achievable, separates otherwise-equivalent screeners by a wide margin, and is only partly visible to fixed perturbation testing. Because the separation is established over an independently audited action space, the result does not rest on transformations whose behaviour we cannot vouch for—and the Tier-C analysis shows the weaker-evidence primitives were never carrying the claim.

Implications for practice: A wallet or screening service should evaluate candidate screeners against a score-aware adaptive adversary restricted to an audited transformation set, and should report robust recall rather than clean recall alone. On this benchmark that criterion selects a different model than clean AUPRC does.

What we do not claim: We claim no architectural novelty; every component is standard. We do not claim clean superiority for hierarchy over a parameter-matched flat encoder, which our own ablation does not support. We do not claim formal semantic equivalence for any transformation. We do not claim robustness against white-box gradient attacks, adaptive retraining, or arbitrary attacker-authored transformations; the claim is scoped to a query-budgeted adaptive adversary over the evaluated transformation space, with Tier B and Tier C reported separately.

A. Limitations

Label provenance. Reference labels encode a decompiler-derived structural condition rather than adjudicated malicious

intent. The relative, adversarial framing reduces but does not eliminate dependence on that condition: the ranking under attack is a statement about screeners, but clean numbers and the detected-source pool still inherit the rule's definition. Independent human adjudication remains the right next step.

Preservation evidence. The audit is empirical, not a semantic-equivalence proof, and covers a finite calldata suite. It is nonetheless substantially stronger than a spot check: 5,196 audited transformed executions across 120 distinct bytecode families with no observed failure in the three audit-supported classes, including coverage of the inbound hooks the reference rule keys on. Address and selector rewriting remain weakly supported and are confined to Tier C.

Adversary scope. The adversary is score-access, bounded-overhead, and restricted to the evaluated action space at a 64-query budget with at most four composed actions. White-box and higher-budget adversaries are unevaluated. AuthGuard-Seq's residual Tier-B ASR of 0.197 means roughly one in five detected risky delegates is still evadable within our own threat model.

Replication depth. AuthGuard-Seq and the emulator—the central comparison—were evaluated at three seeds. The flat convolutional and gradient-boosted screeners were evaluated at seed 7702 only in the tier analysis, as were the augmentation and parameter-matched blocks. We do not imply identical replication depth across all four models, and do not rank across differing seed scopes.

Live population. The temporal holdout is unlabeled, Ethereum-only, and two weeks long, whereas the benchmark spans seven chains. It measures operating-point transfer, not detection quality, and its 9.3% family-level overlap with training bytecode is measured and reported rather than removed from the headline figure.

Family threshold. The 0.85 MinHash threshold is a design choice. A memorization control shows only 1.2% of test rows have a > 0.9-similar training neighbour, but 43.6% fall in the 0.7–0.9 band, so near-duplicate structure does cross folds below the threshold.

VIII. AI Disclosure

In accordance with venue policy we disclose the role of AI tools in this work. AI coding assistants were used substantively in implementing the experimental harnesses, analysis scripts, and figure generation, and in drafting and editing the manuscript text. Experimental design, tier definitions, the preservation-audit protocol, and all scientific claims were author-directed. No AI system generated, imputed, or estimated any reported numerical result: every value in this paper was produced by executed code over frozen data and model artefacts, recorded in a machine-readable results ledger with seed scope, fold scope, denominator, and provenance, and verified by the authors against that ledger. Frozen-artifact integrity checks were run before and after every experiment. The authors take full responsibility for the contents of the paper.

IX. Conclusion

We evaluated bytecode-only pre-authorization screening for EIP-7702 delegation against an adversary who authors the code being screened. Clean performance alone is insufficient to select a screener: a 15-feature logistic regression is statistically indistinguishable from a hierarchical attention model on unmodified bytecode. Under a query-budgeted adaptive adversary restricted to an independently execution-audited transformation space—flooding, metadata rewrite, and neutral insertion, preserving externally visible behaviour on all 5,196 audited executions across 120 bytecode families—the two separate decisively: a paired difference of $+0.370$ in attack success with 95% CI $[+0.208, +0.520]$ over three seeds, and end-to-end robust recall of 0.679 against 0.378. Expanding to a broader space containing weaker-preservation transformations changes the result only modestly, and parameter-matched controls together with direct attention-mass measurements identify selective aggregation, not capacity, as the associated mechanism. Adaptive, preservation-aware evaluation is therefore not an optional extension for learned security screening over attacker-authored programs; on this task it is what determines which screener one should choose.

References

[1] V. Buterin, S. Wilson, A. Dietrichs, and lightclient, "EIP-7702: Set Code for EOAs," *Ethereum Improvement Proposals*, no. 7702, May 2024. [Online]. Available: https://eips.ethereum.org/EIPS/eip-7702

[2] M. Huang, H. Liu, S. Yang, D. Wu, and S. Wang, "Revealing the Dark Side of Smart Accounts: An Empirical Study of EIP-7702 Incurred Risks in Blockchain Ecosystem," in *Proc. 35th USENIX Security Symposium*, 2026, to appear.

[3] M. Qi, Q. Wang, R. Li, T. Zhu, and S. Chen, "EIP-7702 Phishing Attack," *arXiv preprint arXiv:2512.12174*, Dec. 2025.

[4] P. De Rosa, S. Queyrut, Y.-D. Bromberg, P. Felber, and V. Schiavoni, "PhishingHook: Catching Phishing Ethereum Smart Contracts Leveraging EVM Opcodes," in *Proc. 55th Annu. IEEE/IFIP Int. Conf. Dependable Systems and Networks (DSN)*, Naples, Italy, 2025, pp. 222–232.

[5] W. Wang, J. Song, G. Xu, Y. Li, H. Wang, and C. Su, "ContractWard: Automated Vulnerability Detection Models for Ethereum Smart Contracts," *IEEE Trans. Netw. Sci. Eng.*, vol. 8, no. 2, pp. 1133–1144, Apr.–Jun. 2021.

[6] N. Ashizawa, N. Yanai, J. P. Cruz, and S. Okamura, "Eth2Vec: Learning Contract-Wide Code Representations for Vulnerability Detection on Ethereum Smart Contracts," *Blockchain: Research and Applications*, vol. 3, no. 4, art. no. 100101, Dec. 2022.

[7] Y. Liu, X. Su, H. Wu, S. Li, Y. Cheng, F. Xu, and S. Zhong, "Generic Adversarial Smart Contract Detection with Semantics and Uncertainty-Aware LLM," *arXiv preprint arXiv:2509.18934*, Sep. 2025.

[8] M.-D. Tran-Duong, N. H. Phong, N. C. Thanh, D. M. Trung, T. Truong-Huu, V.-H. Pham, and P. T. Duy, "ContractShield: Bridging Semantic-Structural Gaps via Hierarchical Cross-Modal Fusion for Multi-Label Vulnerability Detection in Obfuscated Smart Contracts," *arXiv preprint arXiv:2604.02771*, 2026.

[9] T. D. M. Dai, T. H. M. Le, M. A. Babar, V.-H. Pham, and P. T. Duy, "ORACAL: A Robust and Explainable Multimodal Framework for Smart Contract Vulnerability Detection with Causal Graph Enrichment," *arXiv preprint arXiv:2603.28128*, 2026.

[10] M. Allamanis, "The Adverse Effects of Code Duplication in Machine Learning Models of Code," in *Proc. ACM Onward!*, 2019, pp. 143–153.

[11] F. Pendlebury, F. Pierazzi, R. Jordaney, J. Kinder, and L. Cavallaro, "TESSERACT: Eliminating Experimental Bias in Malware Classification across Space and Time," in *Proc. 28th USENIX Security Symposium*, Santa Clara, CA, USA, 2019, pp. 729–746.

[12] J. Bu, W. Li, Z. Li, Z. Zhang, and X. Li, "SmartBugBert: BERT-Enhanced Vulnerability Detection for Smart Contract Bytecode," *arXiv preprint arXiv:2504.05002*, Apr. 2025.

[13] B. Wang, Y. Tong, S. Ji, H. Dong, X. Luo, and P. Zhang, "A Review of Learning-Based Smart Contract Vulnerability Detection: A Perspective on Code Representation," *ACM Trans. Softw. Eng. Methodol.*, vol. 35, no. 6, 2026.

[14] N. Grech, L. Brent, B. Scholz, and Y. Smaragdakis, "Gigahorse: Thorough, Declarative Decompilation of Smart Contracts," in *Proc. IEEE/ACM 41st Int. Conf. Software Engineering (ICSE)*, Montreal, QC, Canada, 2019, pp. 1176–1186.

[15] P. Tsankov, A. Dan, D. Drachsler-Cohen, A. Gervais, F. Bünzli, and M. Vechev, "Securify: Practical Security Analysis of Smart Contracts," in *Proc. ACM SIGSAC Conf. Computer and Communications Security (CCS)*, Toronto, ON, Canada, 2018, pp. 67–82.

[16] F. Pierazzi, F. Pendlebury, J. Cortellazzi, and L. Cavallaro, "Intriguing Properties of Adversarial ML Attacks in the Problem Space," in *Proc. IEEE Symp. Security and Privacy (SP)*, San Francisco, CA, USA, 2020, pp. 1332–1349.

[17] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On Calibration of Modern Neural Networks," in *Proc. ICML*, 2017, pp. 1321–1330.